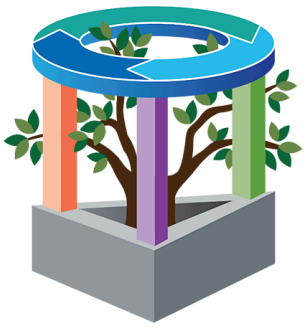
# Drone Based Mapping and Ground Based Robotic Precision Spray System for Field Crops

## Project Requirements and Specifications

AgAID



AgAID

AI Institute for Transforming Workforce & Decision Support

DRONE BASED DEVELOPMENT

Gil Rezin

Logan Sutton

[December 3 2025]

# TABLE OF CONTENTS

# I. Introduction

## I.1 Project Introduction

Andrew Nelson, a fifth-generation farmer and former software developer, uses and implements advanced tech solutions throughout his Palouse farm. These solutions, from Raspberry Pis in grain silos to heavy-duty spraying drones, help Andrew lower labor costs and manage his farm efficiently. One of Andrew's goals is to automate the spraying of a 20-acre wheat field as a proof of concept for future automated pesticide application. Using drone imagery and machine learning, Andrew aims to generate an AI-based prescription and path for a spraying robot, enabling the robot to perform optimal variable rate spraying on his field.

## I.1 Background and Related Work

Precision agriculture has emerged as a valuable strategy for addressing problems in modern farming. This solution allows for responses to in-field variability, minimizing expenditure of critical resources [1]. Some typical precision agriculture solutions include drone sprayers, variable rate technology, and tractor guidance. Our solution seeks to maximize pesticide application efficiency, both with variable rate controlling and optimal path finding. This will be implemented by building upon the concept of drones in agriculture, along with robotic sprayers.

The motivation behind this project also involves entry into the Farm Robotics Challenge. This challenge allows college students to tackle real-world agricultural challenges with robotic solutions [2]. There is a $50,000 SAFE investment as a prize for the top-performing team. Andrew Nelson is the sponsor for the WSU-based team.

The required tech stack involves knowledge of Python, Pix4d, and WebODM. Pix4d and WebODM are professional drone mapping software, which we will use to stitch together aerial images into orthomosaic maps. WebOdm is the open-source counterpart to Pix4d, and aligns better with our goal of providing open-source software to farmers. Both are accessible for our development needs, but knowledge of WebODM will be critical.

## I.3 Project Overview

Farmers face increasing challenges in pesticide application due to labor shortages, increasing input costs, and limitations of existing equipment. Drone sprayers, though promising, are hindered by inaccuracy, require frequent refills, and are not able to fly effectively in windy conditions. These inefficiencies, coupled with increased exposure to chemicals, make drone spraying non-optimal. The need for a practical and accessible solution is imperative.

The goal of this project is to utilize machine learning to automate and optimize pesticide application. We will use drone imagery and imagery analysis tools, open source software, and a custom robotic sprayer to apply pesticides in a test field effectively and efficiently. Specific requirements include:

- Use drone imagery and AI to build prescriptions for spraying
- Transfer prescriptions to the robot sprayer for variable-rate application
- Vary the spray rate across the boom using individual nozzle controls
- Validate effectiveness and determine optimal boom width

The solution to this problem should utilize open-source, locally run software for ease-of-use and be designed with larger, farm-ready robots in remote locations. Additionally, the final product needs to be an accessible system utilizing off-the-shelf components.

Our project's analysis will be conducted over a 20-acre wheat field located in Farmington, WA. The objective is a data analysis and optimization problem: find the minimal amount of pesticide that will cover the field's crops while ensuring complete field coverage (current application levels use 8 refills of pesticide for full coverage). Prior data collection has been conducted via drone, obtaining detailed classifications for each level of pesticide application necessary per measured cell.

Our desired output is as follows: Given an input of data about the field, return a list of instructions for the sprayer robot to follow, including timestamp-specific navigation and individualized nozzle control for fully autonomous, optimized pesticide application.

There are a number of specialized libraries and tools utilized for this project. Pix4d is a photogrammetry software suite designed for drone mapping and visualization, particularly for agricultural purposes. This is our main data container around which the model will be built. WebODM acts as a supplementary program for drone mapping and image processing. The farm robot is supplied by farm-ng: it will act as the autonomous vehicle from which commands generated by the model are executed, and a base from which our hardware team will be working on over the course of the year.

Aside from our previously collected drone data, there is no prior codebase to build from. Due to competition restrictions, no existing codebase for a given project is permitted. Everything in our codebase will be written by us.

### I.4 Client and Stakeholder Identification and Preferences

Jordan Jobe - project coordinator for AgAID & Project Sponsor, who manages the AgAID program and our project communications with clients. Expects project completion in a swift and timely manner before the Farm Robotics Challenge.

Andrew Nelson - Owner of Nelson Farms & Mentor, who owns our testing grounds and gathers drone data for the project. By reducing labor and input costs on his farm, he represents future product users.

Lav Khot - WSU Associate Professor in the Department of Biological Systems Engineering, project overseer. Expects the project to be utilized in future academic research.

Dattatray Ganesh - WSU Graduate Student, assistant project team coordinator. Expects the project to be utilized in future academic research.

## II. Team Members

### II. 1 Logan Sutton

Logan served as a backend developer responsible for implementing core services that support image stitching, field map generation, and data storage. Their work involved learning and applying FastAPI for backend endpoints, understanding the NodeODM API and the PyODM library to manage stitching jobs, and using Docker to run and control the NodeODM containerized environment. They also contributed to the project's path-planning module and played a key role in integrating backend functionality with the frontend, enabling seamless workflows for image stitching, field map viewing, and result retrieval. Through working with these technologies, this developer gained experience with containerized services, job-status polling, and the service interfaces required to support the broader DBD system.

### II. 2 Gil Rezin

Gil has focused on the field-map generation and prescription pipeline, working with clustering methods such as DBSCAN, vegetation-index processing through FIELDimageR, and experimentation with techniques for calculating and grouping NDVI values from orthophotos. He has also contributed to the path-planning module, helping define data formats and outputs compatible with the project's UI components. In addition to his technical responsibilities, he played a key role in communicating with the client and stakeholder to clarify requirements, validate expectations, and ensure the prescription-generation workflow aligned with real-world needs.

## III. System Requirements Specification

### III.1 Use Cases

**Use Case 1: Upload and Process Drone Imagery**
One of the primary use cases for this system is the upload and processing of drone imagery by the farmer. After completing a drone mapping over the field, the farmer collects raw image data that captures the conditions of their crop. Our system provides a simple and intuitive interface where the farmer can click an "Upload" button to add these images to the application, as shown in Figure III.1. Once the images are uploaded, the system validates the files and checks for errors, such as corrupted or mismatched data. After these checks, the images are processed into a stitched field map. This map is the foundation of our processing pipeline, as all further path-finding and robot instructions are based on the map. By supporting multi-file upload, the farmer can easily transition from gathering drone imagery to starting the stitching process. See Table III.1.1 for details.

**Table III.1.1: Upload and Process Drone Imagery Use Case**

| Use Case | Upload and Process Drone Imagery |
|---|---|
| **Actors** | Farmer |
| **Pre-condition** | Farmer has completed a drone mapping session and collected raw image data |
| **Post-condition** | Images are successfully uploaded, validated, and processed into a stitched field map. |
| **Main Flow** | - Farmer clicks the “Upload” button to add drone images.<br>- System validates uploaded files and checks for bad data<br>- Valid images are processed into a stitched field map.<br>- System notifies the farmer when processing is complete. |
| **Related Requirements** | FR-01, FR-02, FR-11 |

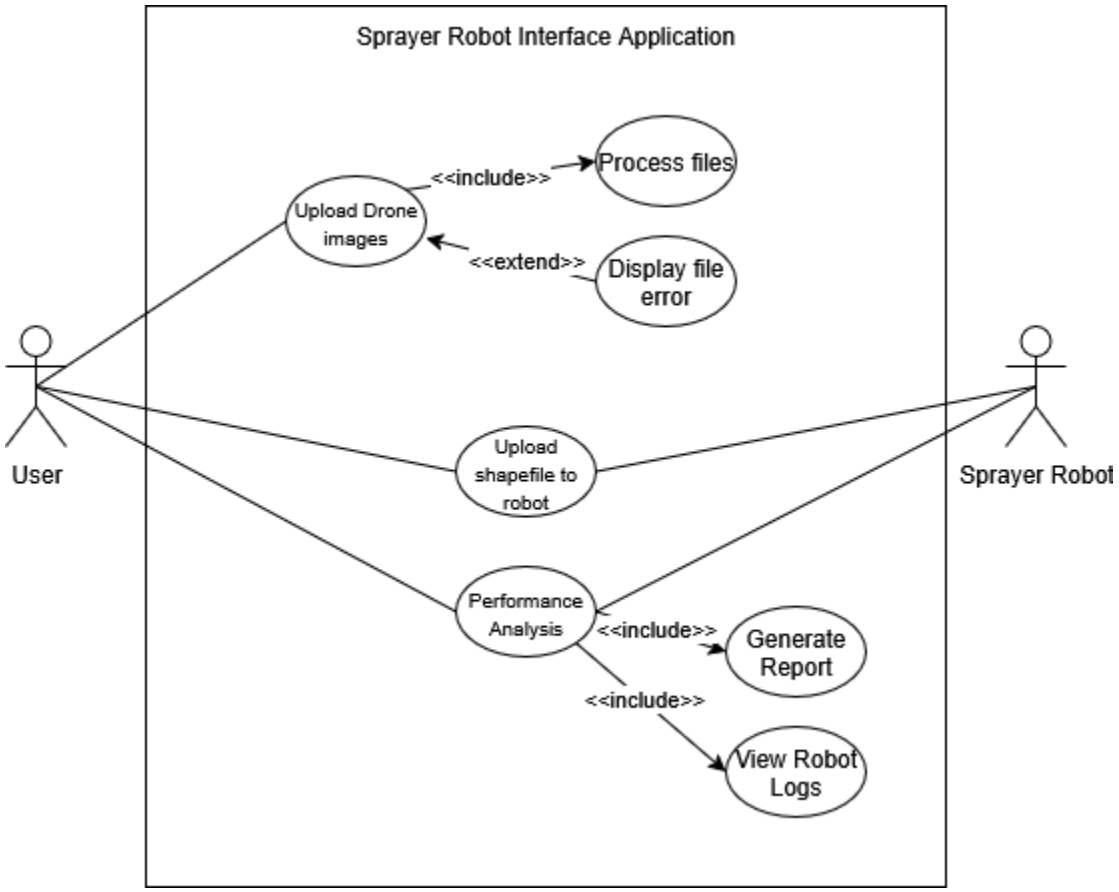



Figure III.1 Application Use Case Diagram

**Use Case 2: Calculate and Execute Optimal Spraying Path**
Another important use case is the calculation of an optimal spraying path and execution by the robot sprayer. The system takes the pesticide prescription and computes the most efficient and safe way to traverse the field. Obstacles such as telephone polls or steep hills are considered.

Before execution, the farmer is given the option to preview the path overlaid on the field map, allowing for a visual inspection of the system's decisions. The farmer will also have the option to auto-deploy if they deem an inspection unnecessary. Once approved, the prescription and path are uploaded to the robot, where the robot sprayer will control individual nozzles based on the prescription and current progress on the spraying path. See Table III.1.2 for details.

**Table III.1.2: Optimal Spraying Path Use Case**

| **Use Case** | **Calculate and Execute Optimal Spraying Path** |
|---|---|
| **Actors** | Farmer, Robot Sprayer |
| **Pre-condition** | Field map and pesticide prescription are available. System is connected to the robot sprayer. |
| **Post-condition** | Robot sprayer completes the spraying operation following the optimized path based on the prescription. |
| **Main Flow** | - System calculates the most efficient and safe spraying path, considering obstacles (e.g., telephone poles, steep hills).<br>- Farmer previews the generated path overlaid on the field map.<br>- Approved prescription and path are uploaded to the robot sprayer.<br>- Robot sprayer executes the operation, controlling individual nozzles according to the prescription and progress. |
| **Alternate Flow** | - System calculates the most efficient and safe spraying path, considering obstacles (e.g., telephone poles, steep hills).<br>- When auto-deploy enabled, the prescription automatically uploads without preview.<br>- Robot sprayer executes the operation, controlling individual nozzles according to the prescription and progress. |
| **Related Requirements** | FR-04, FR-06, FR-07, FR-09, FR-15 |

**Use Case 3: Analyze Spraying Effectiveness**

Finally, the system also supports an analysis of spraying effectiveness. After completing a spraying cycle, the robot logs detailed data to the application, including dosage, time stamps, and geographic coverage. This information is compiled into reports viewable by both the farmer and the development team. The purpose of the reports is not only to verify requirements, but also to provide feedback for our machine learning models for continual improvement. If there are any issues with the spraying cycle, such as missing data or sensor failure, the system will flag the issues for later review. See Table III.1.3 for details.

**Table III.1.3 Spray Effectiveness Use Case**

| **Use Case** | **Analyze Spraying Effectiveness** |
|---|---|
| **Actors** | Farmer, Robot Sprayer |
| **Pre-condition** | Spraying cycle has been completed and the robot has logged operational data. |

| | |
|---|---|
| **Post-condition** | Analysis report is generated and accessible to both the farmer and development team. Any issues are flagged for review. |
| **Main Flow** | - Robot logs spraying data (dosage, time stamps, geographic coverage) to the application.<br>- System compiles data into detailed reports.<br>- Reports are made viewable to the farmer.<br>- System identifies and flags missing or faulty data (e.g., sensor failures).<br>- Reports are used to verify requirements and enhance machine learning model performance. |
| **Related Requirements** | FR-13, FR-14 |

A complete list of Use Case tables is found in *Appendix A.*

## III.2. Functional Requirements

See the following tables for use cases relating to each of our five core modules. Each table includes a description, source, and priority for each functional requirement.

**Table III.2.1 Image Upload Module**

| ID | Functional Requirement | Description | Source | Priority |
|---|---|---|---|---|
| FR-01 | Upload Drone Imagery | The system shall allow users to upload drone imagery captured from the field. This enables the system to acquire raw image data required for generating field maps. | Andrew Nelson (client); end-user need for initiating field data processing. | Level 0 – Essential and required functionality. |
| FR-02 | Stitch Drone Imagery | The system shall automatically stitch multiple uploaded drone images into a unified, georeferenced field mapping. | Derived from user story US-01 and core mapping requirement for field visualization. | Level 0 – Essential and required functionality. |
| FR-11 | Multi-File Uploading and Processing | The system shall support simultaneous uploading and processing of multiple drone images to improve workflow efficiency. | Team requirement to ensure scalability and reduce upload time. | Level 1 – Desirable functionality. |

**Table III.2.2 Prescription Generation Module**

| ID | Functional Requirement | Description | Source | Priority |
|---|---|---|---|---|
| FR-03 | Generate Pesticide Prescription | The system shall analyze stitched field imagery to generate a pesticide prescription that specifies dosage levels for each mapped area. | Andrew Nelson's request for variable-rate spraying and precision agriculture goals. | Level 0 – Essential and required functionality. |
| FR-13 | Machine Learning Optimization | The system shall use machine learning simulations to iteratively learn and improve the optimal pesticide prescription based on historical performance data. | GenAI-assisted brainstorming; team enhancement for intelligent prescription generation. | Level 2 – Extra feature or stretch goal. |

**Table III.2.3 Path Planning and Robot Communication Module**

| ID | Functional Requirement | Description | Source | Priority |
|---|---|---|---|---|
| FR-04 | Calculate Optimal Spraying Path | The system shall calculate an optimal route for pesticide application that minimizes overlap, reduces travel distance, and ensures full coverage. | Andrew Nelson's goal for field efficiency and reduced resource use. | Level 0 – Essential and required functionality. |
| FR-06 | Upload Optimal Path to Robot | The system shall transmit the computed optimal spraying path to the physical robot for execution. | Andrew Nelson; robot control integration requirement. | Level 0 – Essential and required functionality. |
| FR-07 | Start Notification Service | The system shall notify the robot when to begin its pesticide application service once all necessary data has been uploaded. | Derived from workflow automation requirement; ensures synchronized operation between software and hardware. | Level 1 – Desirable functionality. |

| | | | | |
|---|---|---|---|---|
| FR-12 | Upload Prescription Button | The user interface shall provide a dedicated button that allows users to manually trigger the upload of a generated prescription to the robot sprayer. | User story US-02; improves usability and manual control. | Level 1 – Desirable functionality. |

**Table III.2.4 Visualization and Control Module**

| ID | Functional Requirement | Description | Source | Priority |
|---|---|---|---|---|
| FR-08 | View Field Mapping | The system shall allow users to view and interact with generated field maps for verification and analysis purposes. | Stakeholder usability need; supports informed decision-making by the farmer. | Level 0 – Essential and required functionality. |
| FR-09 | Manual Override of Sprayer Settings | The system shall allow the user to manually override the robot sprayer's parameters, such as nozzle rate and start/stop control, during operation. | Farmer user story; ensures safety and adaptability to field conditions. | Level 1 – Desirable functionality. |
| FR-15 | Visual Path Preview | The system shall provide a visual preview of the planned spraying path overlayed on the field map before execution, allowing users to confirm or adjust the route. | GenAI brainstorming suggestion; improves transparency and user trust. | Level 2 – Extra feature or stretch goal. |

**Table III.2.5 Data Management and Reporting Module**

| ID | Functional Requirement | Description | Source | Priority |
|---|---|---|---|---|
| FR-05 | Upload Prescription to Robot Sprayer | The system shall upload the generated pesticide prescription directly to the connected robot sprayer for automated application. | Client requirement for seamless data transfer and automation. | Level 0 – Essential and required functionality. |

| | | | | |
|---|---|---|---|---|
| FR-10 | Data Storage for Reuse | The system shall store all generated field mappings, prescriptions, and optimal paths for later review or reuse. | Stakeholder requirement; supports data persistence and field management history. | Level 1 – Desirable functionality. |
| FR-14 | Application Logging and Compliance Reporting | The system shall automatically log all pesticide applications, including dosage, time, and coverage, to support regulatory compliance and long-term data analytics. | GenAI brainstorming; supports accountability and continual improvement. | Level 2 – Extra feature or stretch goal. |

## III.3. Non-Functional Requirements

**Table III.3.1 Non Functional Requirements**

| ID | Non-Functional Requirement | Description |
|---|---|---|
| NFR-01 | Hardware Performance Requirement | The system shall be capable of running efficiently on a single machine equipped with an Intel i7 CPU (or equivalent) and an Nvidia RTX 3060 GPU (or equivalent). This ensures that image stitching, path generation, and other computation-heavy processes can be performed locally without performance degradation, while remaining affordable for end users. |
| NFR-02 | Image Stitching Time Constraint | The system shall complete the image stitching process for a 20-acre field in no longer than five minutes. This ensures timely generation of field maps, enabling farmers to quickly proceed to prescription and spraying stages. |
| NFR-03 | Data Loading Efficiency | The system shall be able to load previously generated field mappings in under one minute. This improves usability and supports efficient review or re-analysis of prior field data without unnecessary wait times. |
| NFR-04 | Path Calculation Performance | The system shall calculate an optimal pesticide application path in under two minutes for a standard 20-acre field dataset. Meeting this ensures minimal downtime between data processing and field deployment, with longer processing times acceptable for larger datasets. |

| | | |
|---|---|---|
| NFR-05 | System Independence and Offline Capability | The system shall operate fully offline, without reliance on external servers, cloud computing, or internet connectivity. This ensures robustness in rural or low-connectivity environments. |
| NFR-06 | Modular Architecture Design | The system shall be designed using a modular architecture so that core components—mapping, prescription generation, and robot interface—can be independently updated or replaced without impacting other modules. This supports maintainability, scalability, and future integration. |
| NFR-07 | User Interface Responsiveness | The system's user interface shall remain responsive at all times, with no input delays or freezes longer than three seconds during background computations. This ensures a smooth user experience even during intensive processing tasks. |

### III.4 User Stories & Traceability Matrix

The following user stories describe specific actions a user can take in the system, detailing what the user wants to accomplish and why.

**User Story US-01: Upload Drone Imagery**
As an end-user farmer, I want an Upload Drone Imagery button so that I can easily upload my drone's images to generate a field map.

Feature: Upload Drone Imagery
Scenario: Valid imagery upload
Given the user is on the main page of the application
When they press the Upload button
Then they should be prompted to upload their drone imagery files

**User Story US-02: Transfer Prescription to Robot**
As an end-user farmer, I want the system to upload the generated prescription to my spraying robot so that I do not have to manually upload the prescription for each spray.

Feature: Transfer Prescription to Robot
Scenario: Valid prescription upload
Given the user has a prescription generated and ready to upload
When they click Upload Prescription
Then the prescription should be uploaded to the robot sprayer

**User Story US-03: Individual Nozzle Control**
As a sprayer robot, I want to control each sprayer nozzle individually so that I can minimize pesticide resource expenditure.

Feature: Individual Nozzle Control
Scenario: Adjust nozzle spray rates
Given the sprayer robot is moving along the prescribed path

When the prescription specifies different rates for nozzles
Then the robot should adjust spraying rates across the boom accordingly

**User Story US-04: Evaluate Spraying Effectiveness**
As a team, we want to measure the effectiveness of the robotic sprayer so that we can continually improve our ML model.

Feature: Evaluate Spraying Effectiveness
Scenario: Measure and compare results
Given the robot sprayer has completed a spraying cycle
When the team collects the spraying results
Then the results should be compared against the coverage requirements

**User Story US-05 [GenAI]: Visual Path Preview**
As a farmer, I want to preview the spraying path on top of my field map so that I can verify the prescription is correct before sending it to the robot.

Feature: Visual Path Preview
Scenario: Display spraying path overlay
Given a prescription has been generated
When the user clicks “Preview Path”
Then the spraying path should be displayed overlayed on the field map

**Table III.4.1Traceability Matrix**

| Req ID | Requirement Description | Source / Origin | Related Use Case(s) | Related User Story(ies) | Priority |
|---|---|---|---|---|---|
| FR-01 | Upload Drone Imagery | Client (Andrew Nelson) | UC-1: Upload and Process Drone Imagery | US-01: Upload Drone Imagery | Level 0 – Essential |
| FR-02 | Stitch Drone Imagery | Derived from US-01 | UC-1: Upload and Process Drone Imagery | US-01: Upload Drone Imagery | Level 0 – Essential |
| FR-03 | Generate Pesticide Prescription | Client request | UC-2: Calculate and Execute Optimal Spraying Path | — | Level 0 – Essential |
| FR-04 | Calculate Optimal Spraying Path | Client requirement | UC-2: Calculate and Execute Optimal Spraying Path | — | Level 0 – Essential |

| | | | | | |
|---|---|---|---|---|---|
| FR-05 | Upload Prescription to Robot Sprayer | Client requirement | UC-2: Calculate and Execute Optimal Spraying Path; Appendix UC: Transfer Prescription to Robot | US-02: Transfer Prescription to Robot | Level 0 – Essential |
| FR-06 | Upload Optimal Path to Robot | Derived from system design | UC-2: Calculate and Execute Optimal Spraying Path | — | Level 0 – Essential |
| FR-07 | Start Notification Service | Workflow automation | UC-2: Calculate and Execute Optimal Spraying Path | — | Level 1 – Desirable |
| FR-08 | View Field Mapping | Stakeholder usability | UC-1: Upload and Process Drone Imagery | — | Level 0 – Essential |
| FR-09 | Manual Override of Sprayer Settings | Farmer need for control | UC-2: Calculate and Execute Optimal Spraying Path | — | Level 1 – Desirable |
| FR-10 | Data Storage for Reuse | Stakeholder requirement | UC-3: Analyze Spraying Effectiveness; Appendix UC: Visual Path Preview | — | Level 1 – Desirable |
| FR-11 | Multi-File Uploading and Processing | Team scalability requirement | UC-1: Upload and Process Drone Imagery | US-01: Upload Drone Imagery | Level 1 – Desirable |
| FR-12 | Upload Prescription Button | User Story US-02 | Appendix UC: Transfer Prescription to Robot; Appendix UC: Visual Path Preview | US-02: Transfer Prescription to Robot | Level 1 – Desirable |
| FR-13 | Machine Learning Optimization | GenAI brainstorming | UC-3: Analyze Spraying Effectiveness | US-04: Evaluate Spraying Effectiveness | Level 2 – Stretch |

| | | | | | |
|---|---|---|---|---|---|
| FR-14 | Application Logging and Compliance Reporting | GenAI brainstorming | UC-3: Analyze Spraying Effectiveness | US-04: Evaluate Spraying Effectiveness | Level 2 – Stretch |
| FR-15 | Visual Path Preview | GenAI brainstorming | UC-2: Calculate and Execute Optimal Spraying Path; Appendix UC: Visual Path Preview | US-05: Visual Path Preview | Level 2 – Stretch |
| NFR-01 | Hardware Performance Requirement | Non-functional | — | — | Essential |
| NFR-02 | Image Stitching Time Constraint | Non-functional | UC-1: Upload and Process Drone Imagery | — | Essential |
| NFR-03 | Data Loading Efficiency | Non-functional | UC-1: Upload and Process Drone Imagery; UC-3: Analyze Spraying Effectiveness | — | Essential |
| NFR-04 | Path Calculation Performance | Non-functional | UC-2: Calculate and Execute Optimal Spraying Path | — | Essential |
| NFR-05 | System Independence / Offline Capability | Non-functional | All Use Cases | — | Essential |
| NFR-06 | Modular Architecture Design | Non-functional | All Modules | — | Essential |
| NFR-07 | User Interface Responsiveness | Non-functional | All User-Facing Use Cases | US-01, US-02, US-05 | Essential |

### III.5 Standards

Our system adheres to recognized technical and ethical standards to ensure quality, safety, and interoperability throughout the development lifecycle. See Table III.5.1 for details.

**Table III.5.1 Standards and Applications**

| Standard / Code | Domain | Description | Application in Project |
|---|---|---|---|
| IEEE 830 – Software Requirements Specification | Software Engineering | Defines the structure and content of Software Requirements Specification documents. | Used to structure our FRs, NFRs, use cases, and traceability matrix to ensure completeness and verifiability. |
| IEEE 12207 – Software Life Cycle Processes | Software Process Management | Establishes a standard framework for software development and maintenance processes. | Applied to guide our workflow phases (requirements → implementation → verification). |
| W3C WCAG 2.2 – Web Content Accessibility Guidelines | User Interface Design | Ensures accessibility for users with visual or motor impairments. | Used in the design of the visualization dashboard to maintain high-contrast maps and keyboard navigation support. |
| GDPR (2018) – General Data Protection Regulation | Data Privacy and Ethics | Governs the collection and handling of user data. | Applied through explicit consent prompts and anonymized logging of field data; no personally identifiable information is stored. |
| ACM Code of Ethics | Professional Conduct | Promotes fairness, transparency, and user welfare. | Referenced when defining data-use boundaries and ensuring that AI-based prescriptions are ethically generated. |

Narrative:
The IEEE 830 standard guided the creation of a traceable, structured SRS document linking each requirement to its corresponding verification test. WCAG 2.2 informed the design of user interfaces to ensure accessible map interaction and clear visual indicators. Since our system may handle location or field data, we adhered to GDPR principles of data minimization and explicit consent. To maintain professionalism and ethical responsibility, the ACM Code of Ethics was referenced in algorithm transparency and user safety considerations.

**Constraints in Project Solution**
The system was designed under several practical and ethical constraints that shaped major design and engineering decisions. See table III.5.2 for our constraint details.

**Table III.5.2 Constraints and Design**

| Constraint Type | Specific Limitation | Design Trade-off / Adaptation |
|---|---|---|
| Budget Constraint | Approximately zero expenditures on software side | Leveraged open-source frameworks (OpenCV, Python, C#, PySpark) and local computation instead of paid cloud solutions. |
| Hardware Constraint | System must run on an Intel i7 CPU + RTX 3060 GPU | Algorithms optimized for local processing; avoided distributed cloud infrastructure to meet performance within single-PC capability. |
| Schedule Constraint | 30-week development timeline | Adopted iterative Agile sprints; prioritized core modules (image upload, path planning, visualization) and deferred advanced ML optimization to later iterations. |
| Ethical and Legal Constraint | Must comply with data-protection laws | No personal or user-identifiable data stored; all field imagery retained locally and not transmitted externally. |
| Performance Constraint | Path computation ≤ 2 minutes for a 20-acre field | Implemented efficient A* pathfinding and parallel image stitching to meet the target. |
| Usability Constraint | Designed for non-technical farmers | Simplified UI design with large buttons, clear labels, and guided steps for uploading, viewing, and deploying prescriptions. |
| Sustainability Constraint | Minimize energy and compute waste | Chose offline processing and lightweight ML models to reduce unnecessary cloud energy usage. |

**Narrative:**
Budget and time constraints required a focus on essential functionality. The team replaced commercial stitching APIs with custom OpenCV-based solutions, achieving full offline capability within cost limits. The hardware constraint shaped local performance optimizations. Ethical and legal limits prevented storing any private imagery externally, ensuring all processing occurred on the farmer's machine. Collectively, these constraints grounded the project in realistic, sustainable engineering practice.

**Standards and Constraints Verification / Testing**
Verification was conducted to ensure compliance with both documented standards and design constraints. See Table III.5.3 for our verification and testing constraints.

**Table III.5.3 Verification and Testing**

| Verification Type | Standard / Constraint | Method / Tool | Evidence | Result |
|---|---|---|---|---|
| Requirements Structure | IEEE 830 | Manual review of SRS | Traceability Matrix (Appendix C) | Passed – All FRs/NFRs uniquely traceable |
| Lifecycle Compliance | IEEE 12207 | Development phase documentation | Workflow log | Passed – Requirements, design, testing phases aligned |
| Accessibility | WCAG 2.2 | UI audit (contrast & navigation test) | Field map viewer screenshots | Passed – Text contrast ratio > 4.5:1 |
| Data Protection | GDPR (2018) | Code inspection & test logging | No PII storage logs | Passed – No external data transmission |
| Performance | Path computation ≤ 2 min | Simulation test (20-acre dataset) | Performance log | N/A |
| Hardware Feasibility | i7 / RTX 3060 local runtime | Local benchmark test | Resource usage log | Passed – CPU/GPU load < 70% |
| Budget | ≤ $100 | Expense documentation | Equipment purchase record | Passed – $0 total |
| Sustainability | Offline operation | System execution test | Local execution proof | Passed – Fully functional without internet |

**Narrative:**
All major standards were verified through targeted testing. Documentation compliance with IEEE 830 and IEEE 12207 was confirmed via internal review and traceability mapping. The UI satisfied WCAG 2.2 Level AA accessibility criteria. Data-handling audits confirmed GDPR compliance, ensuring privacy and ethical operation. Hardware and performance tests validated real-time functionality within defined resource limits, while cost tracking confirmed budget adherence. Overall, all standards and constraints were met or exceeded, affirming system readiness and professional engineering rigor.

# IV. System Evolution

The optimal path-finding requirement may not be needed if the engineers building out the robot decide to use a John Deere navigation system. The John Deere system would allow for a straight upload of a field map prescription and automatically find a path for the robot sprayer to traverse. If this is the case, then the need for a Machine Learning algorithm to determine the spraying path becomes obsolete. This decision will not be transparent until the engineers determine which system would work best for our project. At this time, we are considering both options and find it valuable to explore generating our own optimal path as an opportunity to learn. The tradeoffs for this choice are found in Table IV.1.

**Table IV.1 ML Path VS John Deere Pathing**

| | Pros | Cons |
|---|---|---|
| Machine Learning Optimal Path Generation | - No extra costs/fees<br>- Great learning opportunity<br>- Would be free software for Farmers | - Extra time to develop<br>- Require additional computational resources |
| John Deere System | - Already existing, working system<br>- Has steering and navigation control built in<br>- Easily upload the generated field map prescription | - $10,000 for sensors, licensing, etc.<br>- It may be difficult to integrate with a spraying robot<br>- Sploofing could be required |

We also anticipate that as the system grows, there may be a need for additional computational power. Initially, this project is focused on stitching imagery for small fields (approximately 20 acres). Farmers have a wide range of field sizes, some fields spanning up to 1,000 acres. If we want this application to be scalable to large farms, the PC specifications required to run our system will also increase. For reference, our client has trouble stitching imagery from his 700-acre field on a very high-end PC with 128 GB RAM. We don't believe we will be stitching imagery for fields this large, but it is an implication to consider nonetheless.

# V. Architecture Design

The DBD system is built as a fully offline, modular desktop application, ensuring full operability in rural areas without internet access. The system is designed modularly, utilizing various pieces of software in unison to provide a fluid and intuitive interface for the farmer end users. The modules are divided based on our pipelined flow of data, and also adhere to our system goals, which are listed below.

The primary system goals are to:

- Reduce pesticide and labor costs through automated, variable-rate spraying.

- Generate optimal application routes that avoid hazards such as steep hills.
- Operate autonomously without reliance on external cloud infrastructure.
- Maintain modularity and scalability for integration with other systems (e.g., John Deere navigation).

Each module operates independently and can be upgraded or replaced as technology advances. The five core modules are:

1. **Image Stitching Module**
   This backend module handles the import and validation of drone imagery and also handles orthophoto creation. The orthophotos are generated using a locally run image of NodeODM via Docker. This provides a free and easy-to-use image stitching functionality.
2. **Prescription Generation Module**
   This backend module produces AI-based dosage maps (field maps) from stitched imagery. These maps are produced using an R orthophoto analysis library called FIELDimageR - by calculating and then analyzing the per-pixel NDVI values with unsupervised clustering, an individual plant's health can be determined and a pesticide amount is assigned.
3. **Path Planning and Robot Communication Module**
   This backend module produces a series of geographic waypoints for the robotic sprayer to follow and execute actions along. FarmNG provides the utilities to send these instruction files to the sprayer robot. It is not yet determined whether the sprayer robot has the functionality to automatically calculate a path given a field map, so the exact implementation is still not certain.
4. **Visualization and Control Module**
   This frontend module provides the user interface for visualization and user input. The interface is managed using React for a fast and simple locally hosted application. React provides all the necessary tools for providing visualization and image upload functionality, and is also simple to implement with AI tools. A simple user interface is ideal, as the user is intended to only have to upload drone imagery, and the entire system will autonomously complete its pipelined tasks without additional instruction.
5. **Data Management and Reporting Module**
   This backend module stores, logs, and reports field operations for compliance and review. We will be using a determined local file structure to accomplish this task, as we deemed using a locally run database was unnecessary and may cause slower operations with the transferring of images.

All backend modules are served from a common API layer, which distributes the necessary information to each responsible location. Additionally, the API is set up to serve the frontend React application via a series of robust endpoints. This provides a scalable and maintainable system that works entirely offline. The backend API layer is implemented using Python, specifically FastAPI.

All system computation will occur locally on a workstation meeting the following minimum requirements:

- Intel i7 CPU (or equivalent)
- NVIDIA RTX 3060 GPU (or equivalent)

These specifications ensure the system performs complex computations—such as stitching, machine learning inference, and path optimization—within acceptable time limits.

## V.1. Overview

The DBD system architecture is designed around modularity, local computation, and offline autonomy. Its structure follows a layered, service-oriented model in which all modules interact through a unified local API. This approach isolates responsibilities, simplifies maintenance, and ensures that upgrades to individual components, such as new AI models or imaging pipelines, can be integrated with minimal disruption to the broader system.

At the center of the architecture lies the FastAPI-based backend layer, which acts as a communication hub between processing modules and the user interface. Each backend service registers its own endpoints through this API, enabling seamless data flow between the front-end React application and the offline computational modules. This design provides a consistent, well-defined interface for data exchange while preserving the independence of each processing stage.

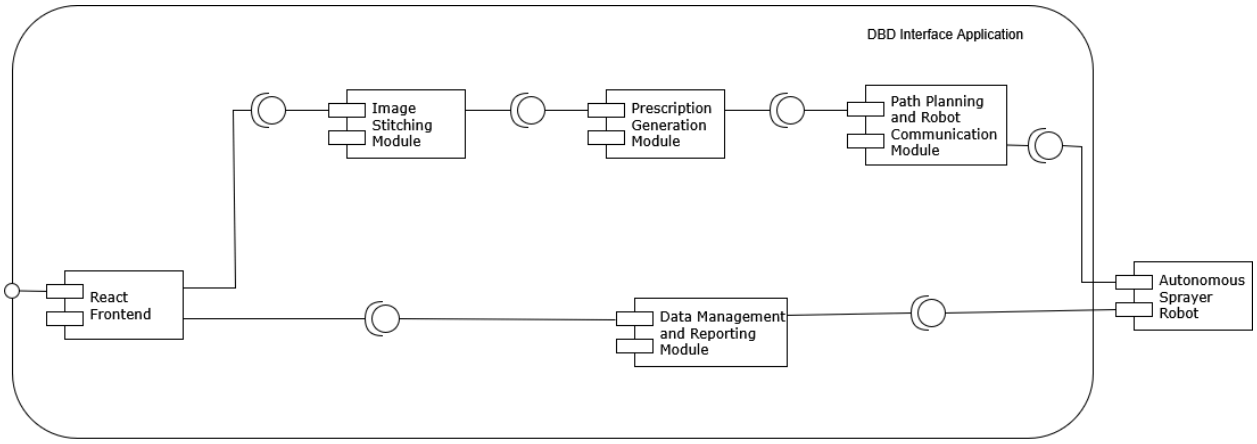
The decision to adopt an entirely local, offline architecture was driven by the system's target environment: rural agricultural areas with unreliable or nonexistent internet connectivity. By running all modules on a single workstation, the system ensures deterministic performance, data privacy, and full operability in the field without cloud dependencies. Additionally, this eliminates recurring costs associated with remote hosting or storage while maintaining high processing throughput through GPU-accelerated local computation.

The pipeline architecture reflects the natural data flow of the system, from raw imagery ingestion to output visualization, while remaining flexible enough to accommodate new components in the future (for instance, automated calibration or external database integration). The modular separation also enables independent development and testing of each component, supporting long-term scalability.

From a deployment standpoint, all computational modules are containerized where applicable (e.g., image processing within NodeODM's Docker instance) to ensure portability and reproducibility across different hardware setups. The front-end React interface is served locally through the same FastAPI framework, creating a cohesive, self-contained desktop environment that requires no external hosting or network services.

Figure V.1.1 illustrates the overall system architecture, showing the interaction between the frontend interface, the centralized API layer, and the offline backend modules within the local computing environment.

**Figure V.1.1: Component Diagram**




**Workflow Summary:**

1. Drone captures raw imagery of the field.
2. Images are uploaded and stitched into a georeferenced field map.
3. The field map is analyzed to generate a dosage prescription map.
4. The system computes an optimized path for the robot sprayer.
5. The prescription and path are transmitted to the robot for execution.
6. Operational data are collected and analyzed to evaluate spraying effectiveness.

## V.2. Subsystem Decomposition

### V.2.I Image Stitching Module

**Description:**
The Image Stitching module handles the ingestion, validation, and stitching of aerial drone imagery. The stitched output serves as the foundational data layer for subsequent modules. To achieve this task, we utilize NodeODM, a free and open-source image stitching software that can run locally and offline. NodeODM is used to generate the orthophoto and relevant data for the Prescription Generation Module.

**Concepts and Algorithms:**

The Image Stitching module handles several tasks. First, the module inputs several drone multispectral images that will all be combined to form a single image. Each image will be validated for correct file formats and similarities to ensure that every file can be merged into a single image representing the entire field. This process is done locally, as our program must be used in remote farms without access to high-speed internet. The module also provides a method for retrieving the processed data.

**Services Provided**

| Service Name | Provided To | Description |
|---|---|---|
| Put Drone Imagery | Backend API | Receives raw drone images captured from farmer's field and prepares them for processing within the NodeODM environment. |
| Validate Imagery Input | Backend API | Ensures that uploaded drone imagery meets the required format, resolution, and metadata standards before processing. |
| Generate Orthophoto | Backend API | Uses the NodeODM engine to stitch individual drone images into a single high-resolution orthophoto (georeferenced map). |
| Get Processed Data | Backend API | Packages and returns processed outputs (orthophotos, DSMs, and metadata) to the backend API for downstream processing. |

**Services Required**

| Service Name | Provided By | Description |
|---|---|---|
| Drone Imagery Upload | Farmer's Drone / Field Interface | Provides the raw imagery data required for processing. |
| Prescription Generation Module | Backend API | Consumes the orthophoto and derived outputs from NodeODM to produce AI-based dosage or field maps. |

### V.2.II Prescription Generation Module

**Description:**
This module analyzes the field map to generate a pesticide prescription that defines dosage levels for specific areas of the field. The prescription is generated utilizing machine learning techniques, specifically DBScan clustering, to identify proper spraying locations and optimal pesticide application amounts.

**Concepts and Algorithms:**
The Prescription Generation module utilizes a machine-learning classification model to predict dosage of pesticide per subdivision of the field map. This is done through NDVI scores, metrics that measure plant health from light reflectivity, analysis done via the R library FIELDimageR. Furthermore, deeper analysis can be done via DBScan clustering. By analyzing the maps

generated by FIELDimageR, machine learning identifies unique clusters of pesticide dosage to be chosen per subdivision.

**Services Provided**

| Service Name | Provided To | Description |
|---|---|---|
| Generate Vegetation Indices | Backend API | Uses the R library FIELDimageR to calculate NDVI values from orthophotos, providing quantitative measures of plant health across the field. |
| Analyze Field Subdivisions | Backend API | Segments the processed orthophoto into smaller field subdivisions to enable localized analysis and dosage prediction. |
| Predict Optimal Dosage | Backend API | Utilizes a trained machine learning classification model to predict the optimal pesticide dosage for each field subdivision based on vegetation indices and historical data. |
| Reinforcement Learning Optimization | Backend API | Applies reinforcement learning techniques to refine dosage recommendations over time, using performance feedback from previous spraying outcomes. |
| Generate Prescription Map | Backend API | Produces a spatially-referenced prescription (dosage) map that encodes the optimal pesticide levels per subdivision, ready for integration with autonomous spraying systems or robotic applicators. |
| Export Prescription Data | Path Planning & Robot Communication Module | Formats and sends the generated prescription map and dosage data for use in path planning and automated field execution. |

**Services Required**

| Service Name | Provided By | Description |
|---|---|---|
| Orthophoto and Vegetation Imagery | NodeODM Module | Provides high-resolution orthophotos and related imagery data used for vegetation analysis. |

| | | |
|---|---|---|
| Historical Spraying Data | Farm Database / API | Supplies prior spraying results and field performance metrics used to train and improve the machine learning dosage prediction model. |
| Reinforcement Learning Feedback | Robot Communication Module | Returns real-world performance data from implemented prescriptions, enabling continuous learning and dosage optimization. |

### III.2.III. Path Planning and Robot Communication Module

**Description:**
This module calculates an optimal route for the ground-based sprayer robot based on the field prescription and terrain constraints, and communicates the resulting path and prescription data to the robot for execution. The results from the Image Stitching module (an orthophoto) are fed in as input. The orthophoto provides geo data that our path planning software can utilize to create a waypoint script for the sprayer robot.

**Concepts and Algorithms:**
The main objective is to calculate the optimal route from an input field Shapefile. Pathfinding is utilized to minimize overlap and travel distance, while maximizing field coverage. This is primarily done through field heading selection, as field coverage is mostly done through straight lines, with the exception of obstacle avoidance, such as telephone poles and difficult terrain. This module communicates directly with the robot's controller to execute the actions generated by this module.

**Services Provided**

| **Service Name** | **Provided To** | **Description** |
|---|---|---|
| Import Orthophoto Data | Backend API | Receives georeferenced orthophotos and associated field shapefiles from the Image Stitching (NodeODM) module for spatial analysis and route generation. |
| Generate Field Coverage Path | Backend API | Computes an optimal traversal path for the sprayer robot based on the field prescription map, terrain constraints, and field geometry, ensuring complete coverage with minimal overlap. |
| Obstacle Detection and Avoidance | Backend API | Identifies obstacles such as poles, uneven terrain, or restricted zones using map data and adjusts the robot's route accordingly to ensure safe navigation. |

| | | |
|---|---|---|
| Generate Waypoint Script | Robot Sprayer | Converts the optimized path into a series of GPS waypoints and commands interpretable by the robot's control system for automated execution. |
| Communicate Prescription and Route Data | Robot Sprayer | Sends the final prescription and waypoint data to the robot's onboard controller, initiating automated field spraying operations. |
| Monitor Execution Feedback | Backend API | Receives updates from the robot regarding task completion, movement accuracy, and spray progress, enabling operational monitoring and future optimization. |

**Services Required**

| Service Name | Provided By | Description |
|---|---|---|
| Orthophoto and Field Map | Image Stitching (NodeODM) Module | Provides the high-resolution orthophoto and shapefile data necessary for spatial route calculation. |
| Prescription Map | Prescription Generation Module | Supplies the dosage map that determines where and how much pesticide to apply along the generated path. |
| Terrain and Obstacle Data | Farm Database / Onboard Sensors | Provides information about terrain elevations, obstacles, or restricted zones that must be considered during route planning. |
| Robot Control Interface | Sprayer Robot System | Receives the generated waypoint scripts and prescription data for autonomous execution and returns operational feedback to the module. |

### III.2.IV. Visualization and Control Module

**Description:**
This module acts as the primary user interface for interacting with the DBD system. It provides visualization of stitched maps, prescriptions, and planned spraying paths. Additionally, this module provides the interface for uploading drone imagery and provides slider options for adjusting various parameters correlated to the desired field map. Farmers can easily view their uploads and get real-time status updates based on the corresponding step in the pipeline that is being completed.

**Concepts and Algorithms:**
This module consists of everything the user interacts with. The UI utilizes component-based architecture with TypeScript for type safety, implementing a state management pattern through React hooks (useState, useEffect, useCallback) to handle complex application state across multiple views including upload, processing, gallery, field maps, and pesticide prescriptions. The file upload system incorporates advanced drag-and-drop functionality using the react-dropzone library, implementing file validation algorithms that check file types (JPEG, PNG, TIFF), size limits (50MB), and generate unique identifiers for each upload session. The real-time processing monitoring employs a sophisticated polling algorithm that automatically queries the backend every 3 seconds to track task status updates, implementing state synchronization between frontend and backend systems through localStorage persistence and custom event dispatching. The application features responsive design algorithms using Tailwind CSS with breakpoint-based layouts, progressive enhancement patterns for offline functionality, and error handling mechanisms with retry logic and user feedback systems.

**Services Provided**

| Service Name | Provided To | Description |
|---|---|---|
| Upload Drone Imagery | Image Upload Module | Provides the user interface for farmers to upload raw drone imagery, including drag-and-drop functionality, file validation (type, size, and format), and unique upload session handling. |
| Visualize Orthophotos and Field Maps | End Users (Farmers) | Displays high-resolution stitched maps generated by NodeODM, overlaid with prescription data and planned spraying paths for intuitive field visualization. |
| Adjust Field Map Parameters | Backend API | Enables users to modify adjustable parameters (e.g., vegetation thresholds or dosage sensitivity) through interactive sliders and UI controls, sending updated parameters to backend modules for reprocessing. |
| Monitor Processing Pipeline | End Users (Farmers) | Provides real-time status updates of each stage in the processing pipeline (image stitching, prescription generation, path planning), using a polling algorithm that queries backend services every 3 seconds. |
| Display Prescription and Path Plans | End Users (Farmers) | Renders the AI-generated prescription maps and optimized spraying routes, allowing users to review dosage distributions and coverage before execution. |
| Manage Application State | Internal Frontend System | Implements advanced state management using React hooks to synchronize data across multiple views (upload, processing, gallery, map, prescriptions) while maintaining local persistence. |

| | | |
|---|---|---|
| Provide Responsive and Reliable UI | End Users (Farmers) | Ensures seamless and adaptive user experiences across devices through responsive design (Tailwind CSS), offline support, and robust error handling with retry and user feedback systems. |

**Services Required**

| Service Name | Provided By | Description |
|---|---|---|
| Orthophoto and Processed Imagery | NodeODM Module | Supplies the stitched and georeferenced map data displayed within the visualization interface. |
| Prescription Map Data | Prescription Generation Module | Provides dosage maps and plant health metrics to visualize AI-generated treatment recommendations. |
| Path Planning Output | Path Planning and Communication Module | Supplies the optimized waypoint routes and spraying paths to be rendered on the user interface. |
| Processing Status Data | Backend API | Returns real-time task updates for synchronization with the frontend's status monitoring and visualization system. |

### III.2.V. Data Management and Reporting Module

**Description:**
This module manages the persistence of operational data, system logs, and generated reports. It ensures reproducibility, accountability, and long-term analysis capabilities. The storage of this data will be handled using a systematic local file structure for ease-of-access and fast operations.

**Concepts and Algorithms:**
The Data Management and Reporting Module handles the data generated by the program through the local file system. Categories of file are sorted by folder within the filesystem, such as logs, reports, and generated prescription maps. When logs are generated by the robot, they are sent wirelessly to this module for storage within the logs folder. Logs contain timestamps, dosage, and GPS information for debugging purposes. When the robot finishes its spraying, a report is generated by this module regarding spray performance and compliance. When requested by the Visualization and Control module, these files become visible to the user.

**Services Provided**

| Service Name | Provided To | Description |
|---|---|---|
| Store Operational Data | Backend System | Manages the structured storage of operational data, including orthophotos, prescription maps, and path plans, within an organized local file directory for quick access and reproducibility. |
| Log Robot Activity | Path Planning & Robot Communication Module | Receives and stores logs transmitted wirelessly from the robot, including timestamped dosage, GPS coordinates, and performance metrics, enabling detailed traceability and debugging. |
| Generate Spray Reports | Visualization & Control Module | Automatically compiles comprehensive post-operation reports detailing spray coverage, dosage accuracy, and compliance information once the robot completes its spraying tasks. |
| Manage File System Organization | Internal Backend System | Maintains a systematic folder hierarchy (e.g., /logs, /reports, /maps) to ensure consistent file organization, fast I/O operations, and simplified long-term maintenance. |
| Provide Report Access | Visualization & Control Module | Supplies user-facing modules with access to stored reports, logs, and historical data upon request, allowing users to view and download past operation summaries. |

**Services Required**

| Service Name | Provided By | Description |
|---|---|---|
| Operation Logs | Path Planning & Robot Communication Module | Supplies real-time or post-operation logs detailing robot activity, GPS traces, and dosage data for storage and later analysis. |
| Prescription Maps and Results | Prescription Generation Module | Provides final dosage maps and related analytical outputs for recordkeeping and inclusion in generated reports. |
| User Report Requests | Visualization & Control Module | Triggers the retrieval and display of reports, logs, and stored data for user viewing and download within the UI. |

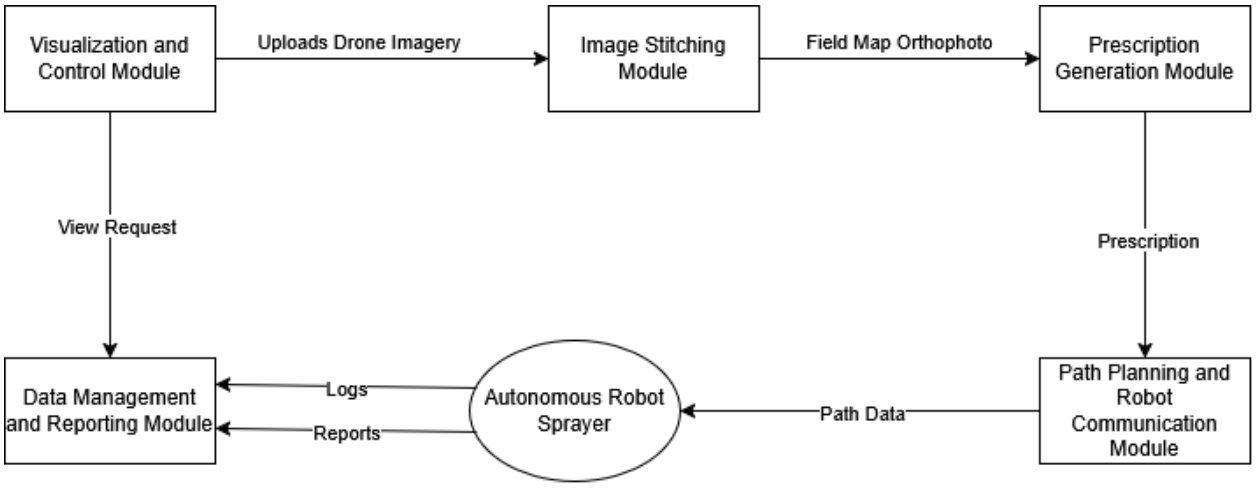
## VI. Data Design

The DBD system handles multiple data types throughout its workflow, including:

| Data Type | Description | Stored In | Example Use |
|---|---|---|---|
| Drone Imagery | Raw aerial photos captured from the field | Local file system, .tif files | Input to stitching process |
| Field Map Orthophoto | Stitched, georeferenced image of the field | Local or cached storage | Input to prescription generation |
| Prescription | Dosage map per field region | Local Shapefile | Input to path planning |
| Path Data | Coordinates for optimized traversal | Local data store | Uploaded to robot |
| Logs | Application and spraying data | Local log files | Compliance and debugging |
| Reports | Post-spray summaries | Local file system | User and regulatory review |

Figure VI.1 illustrates the conceptual data flow between the modules.

**Figure VI.1: Data Flow Diagram for DBD System Components**

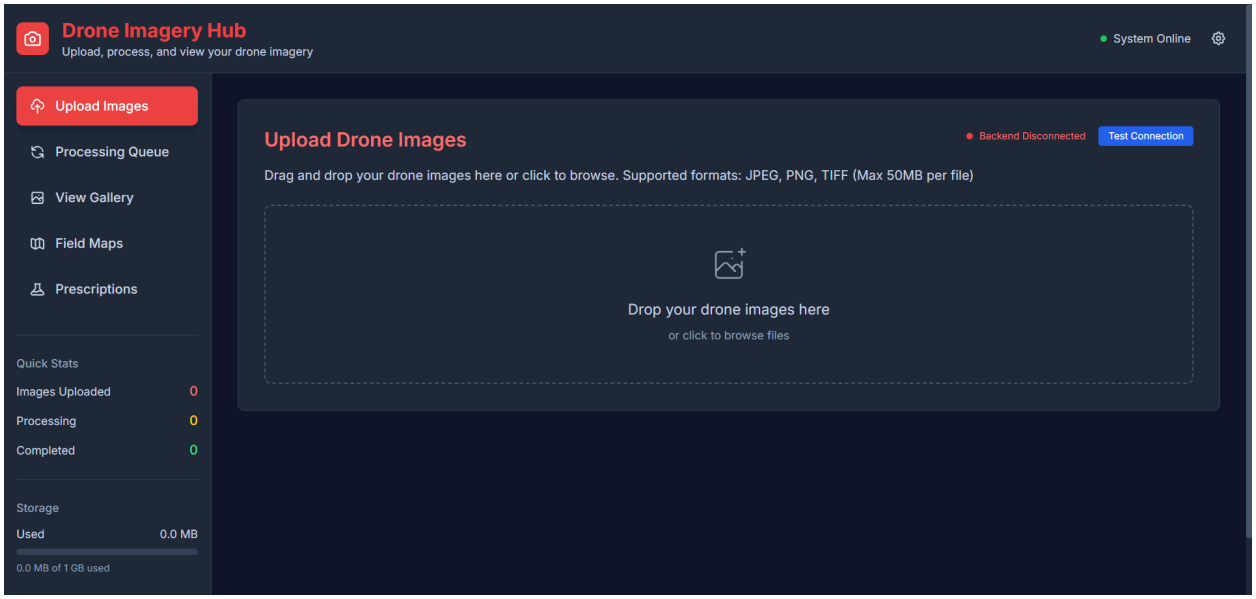



## VII. User Interface Design

The user interface provides an accessible control panel that integrates all stages of the spraying workflow: imagery upload, map generation, path preview, and operation reporting. The design emphasizes simplicity for non-technical users while maintaining transparency in system behavior.

We designed the Drone Imagery Hub frontend with a sophisticated dark theme aesthetic that prioritizes both functionality and visual appeal for agricultural professionals. The interface employs a red and black color scheme (#dc2626 primary red on #0f172a dark backgrounds) to create a modern, professional appearance that conveys the technical nature of drone imagery processing while maintaining excellent readability and contrast. The application utilizes Inter font family throughout for its clean, modern typography that ensures optimal legibility across all components and screen sizes. The component-based architecture implements a sidebar navigation system with five main sections (Upload Images, Processing Queue, View Gallery, Field Maps, and Pesticide Prescriptions) that provides intuitive workflow progression from data input to final agricultural outputs.

**Figure VII.1: Upload Drone Images UI Page (WIP)**




The responsive design employs Tailwind CSS utility classes with breakpoint-based layouts that adapt seamlessly from mobile devices (if a mobile app is wanted in the future) to large desktop displays, ensuring consistent user experience across all platforms. The upload interface features an advanced drag-and-drop zone with visual feedback states, progress tracking bars, and file queue management that provides real-time status updates and error handling. The processing view implements real-time polling algorithms with status indicators and progress visualization that keep users informed of backend processing states. The gallery and field maps sections utilize grid and list view toggles with thumbnail previews and metadata displays that help users quickly identify and select relevant agricultural data. The pesticide prescriptions module incorporates filtering and sorting algorithms with robot compatibility indicators and cost estimation displays that support decision-making workflows. All interface elements feature

smooth transitions (200ms duration), hover effects, and glass morphism styling with subtle borders and rounded corners that create a cohesive, modern aesthetic while maintaining the professional, technical character essential for agricultural drone operations.

## VIII. Constraints

The design of the Drone-Based Detection and Ground-Based Robotic Spraying System is shaped by several technical, environmental, and operational constraints that influence system capabilities and implementation decisions.

**1. Hardware and Performance Constraints**

The robot platform provides limited onboard compute, requiring all intensive processing—image stitching, prescription generation, and path planning—to run on an external workstation with sufficient CPU/GPU resources. Large drone datasets also constrain processing time, requiring stitching and optimization to complete within practical time limits for field operation.

**2. Environmental Constraints**

Drone imagery may vary in quality due to lighting, altitude, and field geometry, affecting stitching reliability. Additionally, real fields contain irregular boundaries and obstacles (e.g., poles, uneven terrain), requiring the path planner to handle complex geometries and safe navigation.

**3. Operational Constraints**

The system must function entirely offline to support remote farm deployment. Robot operation also requires strict safety behavior—including reliable obstacle avoidance and immediate manual override—to prevent crop damage or operator risk. These constraints affect communication protocols and testing procedures.

**4. Data and Software Constraints**

High-resolution drone maps increase data size and memory demands, influencing image handling, storage, and performance. The robot accepts only specific command formats, requiring the system to conform to fixed serialization and interoperability requirements.

**5. Stakeholder and Resource Constraints**

The solution must align with sponsor workflow expectations and Farm Robotics Challenge requirements, shaping UI design and system outputs. Access to farm plots and robot hardware is limited, creating time constraints on real-world testing opportunities.

**6. Budget Constraints**

To keep the system affordable for small farms, the design avoids reliance on cloud services or high-cost onboard processors, favoring open-source tools and offline local computation.

# IX. Project Testing and Acceptance Plan

## IX.1 Overview

This Test Plan will outline the strategy, procedures, and verification methods that will be used to test the Drone-Based Mapping and Ground-Based Robotic Precision Spray System. The system will process drone imagery, generate pesticide prescriptions, compute optimal spraying paths, communicate these paths to a physical robot sprayer, provide visualization and control interfaces, and store operation data for long-term analysis.

Testing will ensure that all major functionalities—including image uploading and stitching, prescription generation, robotics communication, visualization/preview, nozzle control, and reporting—will operate reliably, safely, and according to the system requirements defined in the DBD Cumulative Report. All testing activities will fully utilize access to the physical robot hardware.

### IX.1.1 Test Objectives and Schedule

The primary objectives of this Test Plan will be to:

- Verify that each module satisfies its functional requirements.
- Confirm that non-functional requirements—including performance, responsiveness, modularity, and offline capability—are met.
- Validate all standards and constraints identified earlier in the Requirements and Solution Approach sections.
- Ensure the complete system will operate safely and effectively when integrated with real robot hardware.
- Validate that end-users (farmers, researchers) will find the interface usable, intuitive, and aligned with their workflow.

**Required Resources**
Testing will require access to:

- A drone imagery dataset for a 20-acre field
- The physical robot sprayer
- A controlled outdoor or indoor test environment
- Workstations meeting NFR hardware requirements (Intel i7, RTX 3060 GPU)
- Diagnostic tools, logging utilities, and testing scripts
- Team members with expertise in robotics, backend, ML, and UI

**Schedule Overview (Second Semester)**

- **Week 1–2:** Unit test development and execution for all modules
- **Week 3–4:** Integration testing across all subsystems
- **Week 5–7:** System and performance testing using real hardware
- **Week 8:** Standards & constraints verification
- **Week 9:** User acceptance testing with Andrew Nelson and robotics challenge evaluators
- **Week 10:** Final test reporting and corrections

Deliverables will include test cases, logs, performance metrics, defect reports, verification tables, and a final testing summary. Note: some unit and integration tests have been performed, but the majority have been set for the second semester.

### IX.1.2 Scope

This document will define the full testing process for the DBD system. It will cover unit testing, integration testing, system testing (functional, performance, standards/constraints compliance), user acceptance testing, and environment requirements. The goal is to ensure the complete system meets the expectations for a fully deployable agricultural precision spraying tool.

## IX.2 Testing Strategy

Testing of the DBD system will follow a structured, top-down approach:

1. Unit Testing will be conducted for each module in isolation.
2. Integration Testing will verify interoperability between modules (e.g., stitching → prescription generation → path planning).
3. System Testing will validate system-wide behavior, including robot communication and real spraying operations.
4. Standards and Constraints Verification will ensure alignment with safety, agricultural, software, and ethical standards as documented in the Requirements.
5. User Acceptance Testing will confirm the system meets the needs of the farmer and robotics challenge evaluators.

A Continuous Integration (CI) pipeline will be used for automated unit and integration tests. Although the system will operate offline, CI will help ensure stability with each code update. Continuous Delivery (CD) will not be used because the system is not a continuously deployed web application but rather a standalone local tool with hardware dependencies. Testing results will be documented and reviewed after each major phase.

### IX.3 Test Plans

#### IX.3.1 Unit Testing

Each core module will be tested individually to ensure internal correctness:

**Image Upload Module**

- The team will validate file type checking, image validation routines, multi-file upload logic, and error handling for corrupted images.
- Mock image sets will be used to ensure deterministic behavior before testing with real imagery.

**Prescription Generation Module**

- Unit tests will verify ML model inference, prescription heatmap generation, dosage classification correctness, and error cases including missing stitched maps.
- Synthetic field maps will be used to validate edge cases.

**Path Planning & Robot Communication Module**

- Unit tests will validate pathfinding algorithms, obstacle avoidance logic, and command-packet formation for robot upload.
- Simulator-level tests will validate packet encoding before hardware-level tests begin.

**Visualization & Control Module**

- Unit tests will verify map rendering, zoom/pan interactions, path preview overlays, and manual override UI controls.

**Data Management & Reporting Module**

- Validation will include database operations, log creation, data retrieval, and compliance report formatting.

All unit tests will be executed automatically through the CI pipeline.

### IX.3.2 Integration Testing

Integration testing will validate that modules interact correctly:

1. **Image Upload → Image Stitching → Prescription Generation**
    - Uploaded image batches will be stitched into a map and fed into the prescription generator.
2. **Prescription → Path Planning → Robot Communication**
    - The system will generate a prescription, compute the path, preview it, and transmit it to the robot hardware.
3. **Robot → Data Logging → Reporting**
    - The robot will emit logs during operation, which will be ingested and formatted into a report.
4. **Visualization Layer → All Modules**

    - The visualization module will be tested to confirm correct rendering of stitched imagery, prescriptions, and planned paths.

Integration tests will be executed on progressively larger datasets to ensure scalability.

#### IX.3.3 System Testing

System testing will ensure the full system functions as a cohesive whole, including all hardware interfaces.

##### IX.3.3.1 Functional Testing

Functional tests will be derived from all FRs in the DBD report. Examples include:

- The system will successfully upload, validate, and stitch drone imagery (FR-01, FR-02, FR-11).
- The system will generate an accurate pesticide prescription (FR-03, FR-13).
- The robot will receive and execute an optimal path (FR-04, FR-06, FR-07, FR-12).
- Users will be able to preview field maps and paths (FR-08, FR-15).
- Users will be able to override sprayer parameters during operation (FR-09).
- The system will log all application events (FR-05, FR-10, FR-14).

Each functional test will include expected outputs, acceptance criteria, and test data references.

##### IX.3.3.2 Performance Testing

Performance testing will target non-functional requirements:

- **I**mage stitching time will be verified to complete within 5 minutes for a 20-acre dataset (NFR-02).
- Path calculation will be verified to complete within 2 minutes (NFR-04).
- Data loading of previously generated maps will be verified to complete within 1 minute (NFR-03).
- UI responsiveness will be measured in milliseconds to ensure smooth interactions (NFR-07).
- Offline operation will be tested by removing network access and verifying system functionality (NFR-05).
- Hardware performance will be tested on machines meeting minimum specifications (NFR-01).

Stress tests will include extreme cases, such as excessively large image sets and complex field obstacles.

##### IX.3.3.3 Standards & Constraints Verification / Testing

This section will verify that the system adheres to all standards and constraints defined in the Requirements and Solution Approach.

**Table IX.3.3.3.1: Verification Summary**

| Verification Type | Standard / Constraint | Planned Method / Tool | Planned Evidence | Expected Result |
|---|---|---|---|---|
| Safety | IEEE robotics safety best practices | Physical robot field test + manual override checks | Recorded robot stop events & behavior logs | Robot halts within defined safety window during override |
| Data Integrity | Offline operation requirement | Remove network access during runs | Logs showing all components operate without external connectivity | System functions fully offline (NFR-05) |
| Performance | Image stitching ≤20 min | Timer-instrumented test with real drone dataset | Screenshot/timestamp logs | Stitching completes within 5 minutes |
| Performance | Path calculation ≤2 min | Profiling the pathfinding module on 20-acre dataset | Execution time logs | ≤2-minute completion |
| Hardware Constraint | Target hardware specs | Test on i7/RTX3060 system | CPU/GPU utilization logs | System operates within expected bounds |
| Maintainability | Modular architecture | Static code structure analysis | Module dependency diagrams | Modules update independently without breaking others |
| Ethical / Data | Responsible agricultural AI use | Review prescription logic for fairness & safety | Documentation + test results | No unsafe dosage recommendations |
| System Constraint | Real-world obstacle handling | Robot field test with simulated poles/terrain | Field test video, logs | Robot avoids obstacles 100% of attempts |

| | | | | |
|---|---|---|---|---|
| Usability | Visual path preview clarity | User testing with farmers | Feedback forms | ≥90% task completion rate |

**Verification Narrative**

The team will conduct a full compliance verification process to ensure the system adheres to required industry, ethical, and performance standards. For safety standards, the robot sprayer will be tested in a controlled environment where manual overrides and emergency stop behaviors will be exercised repeatedly. All events will be recorded and analyzed to ensure compliance with safe robotics operation expectations.

Performance standards will be verified using timing instrumentation on representative datasets. Each performance requirement (image stitching, path calculation, data loading, and responsiveness) will be tested under various load conditions to collect evidence showing compliance with NFR-01 through NFR-07.

Constraints identified in the Solution Approach—such as offline operation, hardware limitations, and field environmental conditions—will be tested through structured scenarios. For example, the offline constraint will be validated by ensuring that all major functionalities run without network connectivity. Environmental constraints, such as obstacle-rich fields, will be validated by conducting multiple robot trials under controlled outdoor conditions.

Usability will be verified using user testing sessions with Andrew Nelson and robotics challenge evaluators. Participants will evaluate clarity of the UI, effectiveness of path visualization, and ease of triggering uploads or overrides. Feedback will be logged and used to evaluate user acceptance success metrics.

All verification will produce logs, screenshots, annotated videos, and structured evaluation forms that will be included in the appendix.

**IX.3.3.4 User Acceptance Testing**

User acceptance testing (UAT) will involve actual end-users:

- **Primary User:** Andrew Nelson (farmer)
- **Secondary Users:** Robotics challenge evaluators, team advisors

UAT participants will conduct real workflows, including:

1. Uploading a drone imagery dataset
2. Reviewing the stitched field map
3. Generating and previewing the prescription
4. Reviewing and approving the optimal path

5. Uploading the path to the robot
6. Observing a full spraying cycle
7. Reviewing generated logs and reports

Success criteria:

- ≥90% successful task completion
- No high-severity UI or functionality issues
- Robot performs safely and reliably
- Users confirm system meets workflow expectations

Feedback will be collected via a structured evaluation form.

## IX.4 Environment Requirements

The test environment will include:

- Hardware:
  - Intel i7 or equivalent CPU
  - Nvidia RTX 3060 or equivalent GPU
  - 16 GB RAM minimum
  - Physical robot sprayer with nozzle-level control
- Software:
  - Local workstation environment
  - Python-based backend components
  - ML libraries for prescription generation
  - Logging and profiling tools
- Field Testing Environment:
  - Controlled outdoor test area (e.g., small test plot or open terrain)
  - Safety buffer zones
  - Emergency stop access
- Test Tools:
  - Timer utilities
  - Robot telemetry logger
  - Visualization/UI screen capture
  - Profilers for performance metrics
  - Dataset generators for stress testing

The environment will operate fully offline, consistent with system constraints.

# X. Alpha Prototype Description

The DBD project's alpha prototype implements a functional, end-to-end pipeline for drone image ingestion, stitching via NodeODM, result retrieval, backend processing, and a React-based user

interface. The alpha release focuses on the offline image-processing workflow along with near implementable prescription and path modules:
upload → NodeODM stitching → asset retrieval → gallery visualization → field map generation (WIP) → prescription UI (WIP) → path finding backend (WIP)

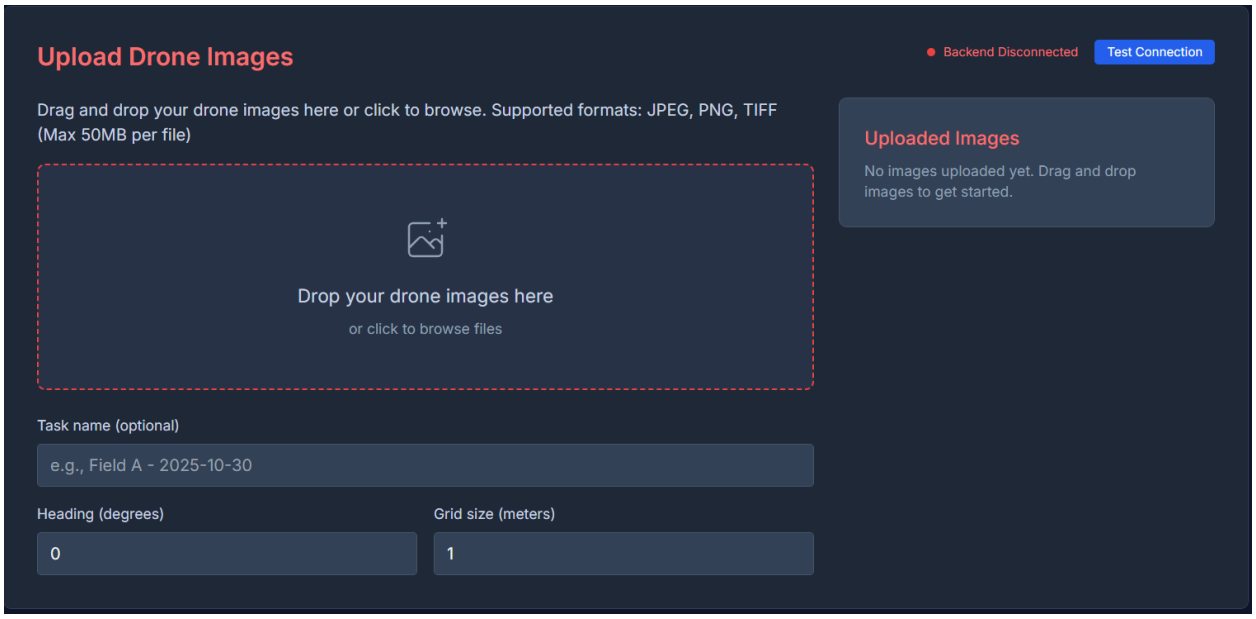
This prototype aligns with the system architecture and functional specifications described in the earlier sections of this report.

## X.1 Image Stitching Module

### X.1.1 Functions and Interfaces Implemented

- Upload Endpoints (FastAPI):
  Accept drone imagery and store files in a local `uploads/` directory.
- NodeODM Integration:
  - Automatic task creation.
  - Status polling until completion.
  - Automated downloading of output assets (orthophotos, PDF reports, metadata).
  - Full pipeline validated when NodeODM is running on `localhost:3000`.
- File-Manifest Storage:
  A lightweight file-based manifest stores metadata for processed results (simple data management solution due to large file sizes).
- Results API Endpoints:
  Endpoints provide download URLs for:
  - Orthophotos
  - PDF reports
  - Task metadata

**Figure X.1 Upload Images UI**




### X.1.2 Preliminary Tests Conducted

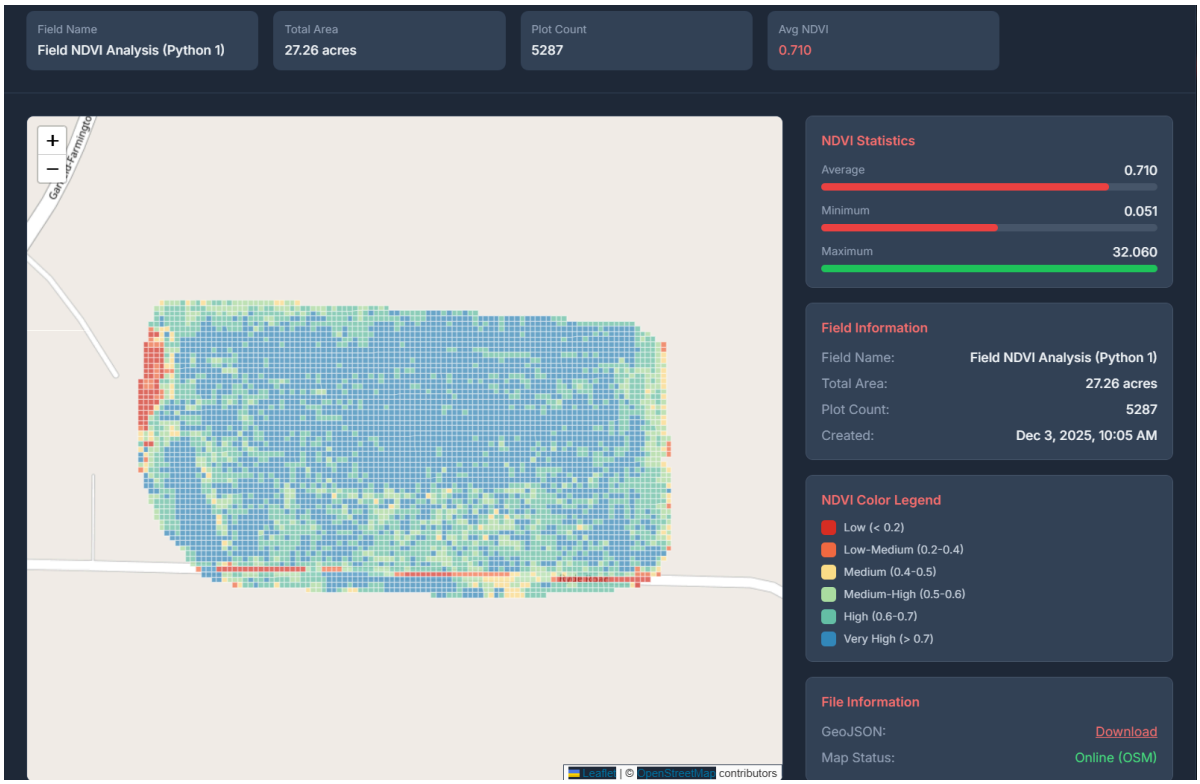
- Backend unit tests (via `pytest`) are set up, though limited.
- Manual validation of NodeODM workflow:
  - Upload → process → download → gallery view.
- Known limitations:
  - No cleanup of failed uploads.
  - Large files may have issues uploading
  - Edge-case handling for unusual NodeODM statuses is incomplete.

## X.2 Prescription Generation Module

### X.2.1 Functions and Interfaces Implemented

- Frontend Prescription UI (Mock Data):
  A fully interactive spray-prescription editor exists, including:
  - Editable grid map
  - Per-cell spray-level controls
  - Visualization tools
- Backend Work in Progress:
  - Branches (`feature/integrate-r-script`) contain full working implementation of R-script for NDVI calculation using FIELDimageR and DBScan for clustering.
  - Backend endpoints for real prescription generation are planned but not yet connected.

**Figure X.2 Field Map Analysis UI (WIP)**




### X.2.2 Preliminary Tests Conducted

- Frontend UI operates as intended.
- Backend prescription generation tests are not yet applicable because the backend is not wired to real vegetation-index computation.
- Mock prescription maps enable UI demo but lack backend support.

## X.3 Path Planning & Robot Communication Module

### X.3.1 Functions and Interfaces Implemented

- Planned Module:
  Architecture and placeholder endpoints exist, but path planning is not yet fully implemented. Currently, the structure that exists is
  - Rough implementation for generating a path for robot to follow based on grid map
  - NDVI values per grid tile with assigned spray type from user
- Prepared Output Structure:
  - The existing data pipeline produces outputs (orthophotos, grid maps) necessary to integrate the path planning module.
  - Have output structure needed for robot to ingest specified path

### X.3.2 Preliminary Tests Conducted

- No hardware tests with the robot platform have been completed.
- Small integration using Farming Python Track Planner example code
- The module remains in the planning stage for next semester.

## X.4 Visualization & Control (Frontend)

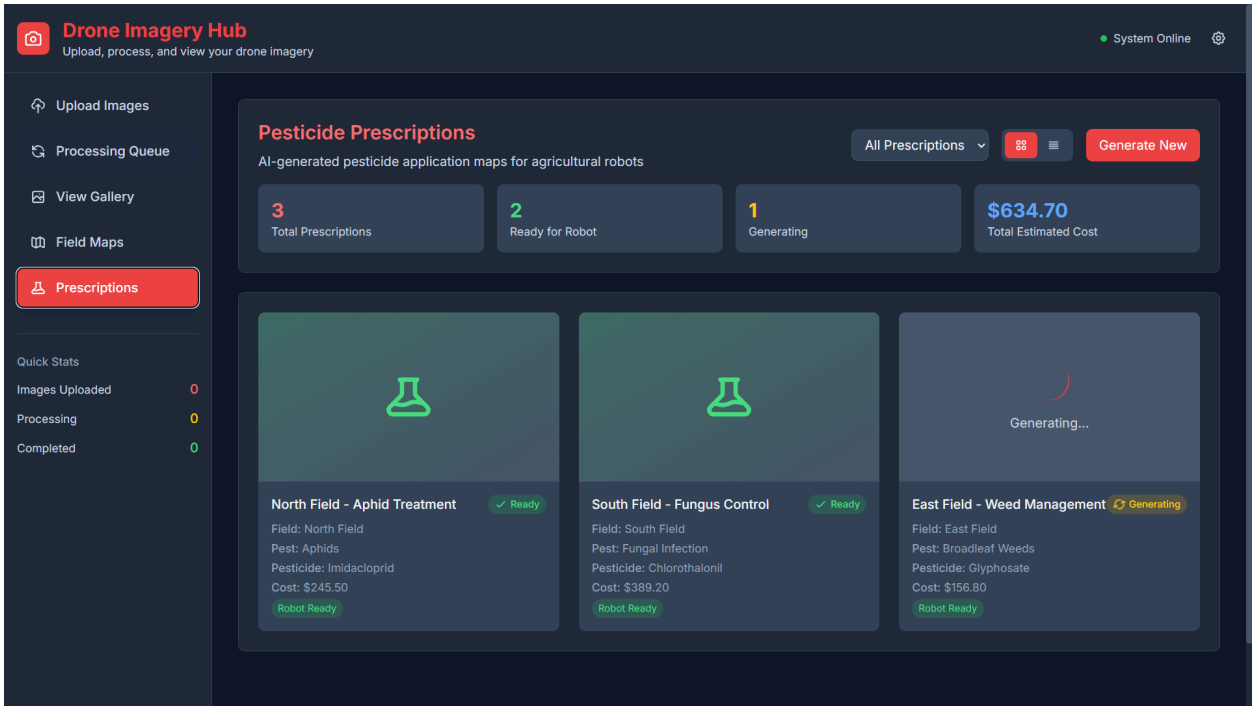
### X.4.1 Functions and Interfaces Implemented

- React Frontend (Tailwind CSS):
  Fully implemented interface, including:
  - Upload View, as shown in Figure X.1
  - Processing View (with automatic status polling)
  - Gallery View
  - PDF viewing
  - Field Maps & Prescription Editor (mock data) as shown in Figure X.2 and Figure X.3
- Processing Polling Logic:
  - Regular status updates for NodeODM tasks.
  - Known bug: certain NodeODM intermediate statuses require a manual refresh.

### X.4.2 Preliminary Tests Conducted

- Manual UI testing confirmed core functionality.
- Gallery/PDF viewer operate correctly with downloaded assets.
- Polling edge-case handling requires improvement.

**Figure X.3 Prescription UI**




### X.5 Data Management and Reporting Module

#### X.5.1 Functions and Interfaces Implemented

- File-based Manifest System:
  Stores metadata about processed tasks.
- Asset Retrieval:
  API returns orthophotos, PDF reports, and task artifacts for frontend display.

#### X.5.2 Preliminary Tests Conducted

- Asset viewing in Gallery verified.
- No persistent database implemented yet.
- CI/CD and automated storage tests planned.

## XI. Alpha Prototype Demonstration

### Demonstration Components

The following will be demonstrated during the Alpha Prototype review:

1. Upload of Multiple Drone Images through React UI.

2. NodeODM Automated Processing, including task creation and job-status polling.
3. Download and Display of Orthophotos & PDF Reports via the Gallery view.
4. Interactive Prescription UI, using mock data to simulate real prescription generation.
5. Demonstrate path generation mock up and detail future improvements

These align with the functional expectations described earlier in this report.

### Mentor Feedback & Planned Improvements

- Integrate real R-based NDVI generation backend.
- Move from file-manifests to a real database.
- Improve file cleanup and error rollback.
- Strengthen NodeODM polling reliability.
- Connect prescription UI to backend service.

## XII. Future Work

The following tasks are planned for next semester:

1. Integrate R-Based Vegetation Analysis
   Complete and merge `feature/integrate-r-script` and produce real prescriptions.
2. Complete Path Planning Module
   - Insert coverage path planning algorithm into automated pipeline
   - Store user specified prescription values based on NDVI ranges
   - Create robot-upload format.
   - Test on physical hardware.
3. Improve NodeODM Error Handling
   - Automatic retries
   - Robust status handling
   - Cleanup of temporary files
4. Large-Scale Dataset Testing
   Validate the full workflow using the 20-acre test dataset

Various branches also exist that are close to being integrated into our main branch, but require finishing touches before a pull request can be made and merged.

## Glossary

**AI-Based Prescription**
A prescription generated using artificial intelligence to determine optimal pesticide dosage levels across different zones of a field.

**Application Logging**
The automated process of recording system operations, data uploads, and field activities to support regulatory compliance and troubleshooting.

**Autonomous Robot Sprayer**
A ground-based robotic system that executes the pesticide prescription by navigating the field and applying chemicals automatically.

**Drone Imagery**
Aerial photographs captured using unmanned aerial drones to assess field conditions and provide input data for stitching and analysis.

**Field Map**
A unified, stitched representation of the farm field created from multiple drone images, used for analysis and prescription generation.

**Image Stitching**
The process of combining multiple overlapping drone images into a single, continuous field map that represents the area accurately.

**Machine Learning (ML)**
A method of data analysis that uses algorithms to learn patterns from previous spraying data and improve future prescriptions.

**Modular Architecture**
A system design in which individual components—such as mapping, prescription generation, and robot communication—can be modified or replaced independently.

**Optimal Spraying Path**
The most efficient route was calculated for the robot sprayer to ensure full field coverage while minimizing overlap and travel distance, while avoiding in-field obstacles such as telephone polls, steep hills, etc.

**Path Planning**
The algorithmic computation of a traversal route for the robot sprayer based on prescription data, obstacles, and field boundaries.

**Pesticide Prescription**
A data-driven map specifying where and how much pesticide to apply across the field, generated from analysis of stitched drone imagery.

**Precision Agriculture**
A farming practice that uses technology and data analysis to optimize resource use, improve yields, and reduce environmental impact.

**Raspberry Pi**
A small, low-cost computer used for automation and monitoring applications, such as sensor control or data logging in farm operations.

**SAFE Investment**
A “Simple Agreement for Future Equity” — a funding mechanism in which investors provide capital with the right to future equity in a project.

**Spraying Cycle**
A complete process in which the robot executes the pesticide application according to the generated prescription and path plan.

**System Evolution**
The expected progression of the system as new technologies, features, or integration requirements are introduced over time.

**User Interface (UI)**
The graphical platform through which the user interacts with the system to upload data, view maps, and control the robot sprayer.

**Variable Rate Spraying (VRS)**
A precision agriculture technique that adjusts pesticide output dynamically based on localized crop needs and prescription data.

**Visualization Module**
A software component that allows users to view, inspect, and confirm generated field maps and spraying paths before execution.

## References

A. Ashworth and P. Owens, “Benefits and Evolution of Precision Agriculture,” *Agricultural Research Service, U.S. Department of Agriculture*, Under the Microscope, January 10, 2025. [Online]. Available: https://www.ars.usda.gov/oc/utm/benefits-and-evolution-of-precision-agriculture/. [Accessed: Sep. 18, 2025]

Farm Robotics Challenge, “The Farm Robotics Challenge,” *UC Agriculture & Natural Resources Innovate*, AI Institute for Next Generation Food Systems, [Online]. Available: https://www.farmroboticschallenge.ai/. [Accessed: Sep. 18, 2025]

# Appendices

## Appendix A

| Use Case | Transfer Prescription to Robot |
|---|---|
| **Actors** | Farmer, Robot Sprayer |
| **Pre-condition** | A valid pesticide prescription has been generated and the robot sprayer is connected to the system. |
| **Post-condition** | The prescription is successfully transferred to the robot sprayer for execution. |
| **Main Flow** | - Farmer selects the option to upload the generated prescription.<br>- System verifies that the robot sprayer is connected and ready to receive data.<br>- Prescription is transmitted to the robot sprayer.<br>- System confirms successful upload and notifies the farmer. |
| **Alternate Flow** | - Farmer attempts to upload the prescription while the robot sprayer is disconnected.<br>- System detects a missing connection and displays an error message prompting the farmer to reconnect before retrying. |
| **Related Requirements** | FR-05, FR-06, FR-08 |

| Use Case | Visual Path Preview |
|---|---|
| **Actors** | Farmer |
| **Pre-condition** | A field map and a generated prescription are available. |
| **Post-condition** | Spraying path is previewed on the field map, verified, and approved (or corrected if invalid). |
| **Main Flow** | -Farmer clicks the “Preview Path” button after a prescription is generated.<br>-System overlays the spraying path on the existing field map.<br>-Farmer reviews the proposed path for accuracy and boundary compliance.<br>-If approved, the farmer proceeds to upload the prescription to the robot. |
| **Alternate Flow** | -System detects that the generated path overlaps with unmapped |

| | |
|---|---|
| | or out-of-bounds regions.<br>- System highlights the invalid areas and prevents upload until corrections are made. |
| **Related Requirements** | FR-10, FR-12, FR-15 |